

小米

算子开发 / GPU Kernel 优化

从语音转写到可复用的面试能力地图

以 CUDA Graph、算子开发流程和性能工程为主，要求候选人能把算子从接口、实现到精度和性能验证完整讲清。

整理口径	1 份记录提炼。内容经语义去噪、合并与重写，不保留逐字稿。
隐私处理	已剔除候选人身份、教育/履历、地点、时间、个人选择、流程沟通与全部反问；不引用原始对话。
阅读方式	先看考察地图定位重点，再按题型练习“定义 - 原理 - 方案 - 验收 - 边界”的闭环表达。

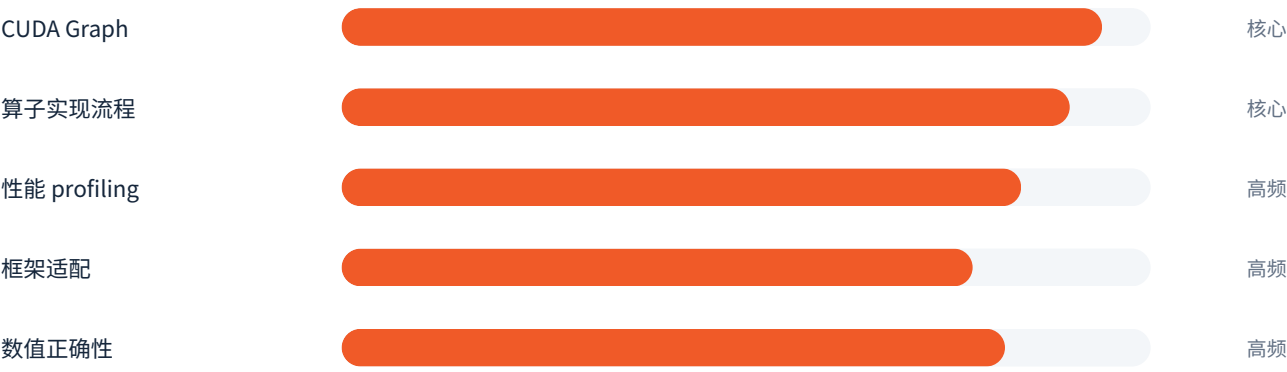
通用高分闭环

任何技术或业务题，都不要直接跳到“方案”。先固定问题边界和度量方式，再展现分析过程、取舍和验收证据。



01 考察地图

下图为材料中反复出现的主题相对覆盖度，用于决定复习优先级，而非量化评分。



面试官真正想验证什么？

面试形态
先用自我介绍和项目筛选方向，再深入问 CUDA Graph 生命周期、kernel 实现和常见调优手段。

实现主线
一个算子应能讲清输入/输出、shape 与 layout、线程映射、访存、同步、数值路径和框架注册。

性能主线
避免只说“换了更快 kernel”。应比较 baseline、瓶颈、优化手段、收益来源和适用 shape。

质量主线
正确性需覆盖容差、边界 shape、随机数据与异常输入；性能需隔离 warmup、异步执行和测量噪声。

答题表达式

问题定义（场景、约束、指标） -> 机制拆解（数据流、资源、依赖） -> 方案选择（备选与取舍） -> 验证（正确性、性能、稳定性） -> 边界（何时无效、如何回退）。

02 高频题型与回答框架

以下不是原始问答转写，而是把多轮追问归纳为可迁移的答题骨架。练习时请替换为可公开、不可识别个人信息的案例。

题型 / 考察意图	可复用回答框架
CUDA Graph 的创建和 replay 有哪些条件？ 考察对运行时约束的掌握。	说明预热、capture、实例化与 replay；强调输入 shape、内存地址、stream/依赖和不支持操作的限制；对动态业务给出分桶、缓存多图与 fallback 的方案。
如何从零实现一个 GPU 算子？ 考察完整开发链条。	先确定数学语义、layout 和精度要求；设计线程块/tile 与内存访问；实现 CPU/GPU 路径和框架注册；最后按正确性、稳定性、性能三组测试逐步验收。
如何定位 kernel 性能不佳？ 考察分析而非调参。	通过 profiler 分出 launch、访存、计算、同步和占用率问题；与理论带宽/算力对比；每次只改一个变量验证，并保留不同 shape 的性能曲线。
算子融合何时不值得做？ 考察边界意识。	当寄存器压力降低 occupancy、融合后 shape 泛化差、调试/回退成本高，或端到端关键路径不在此处时，应优先采用更小粒度优化或保留独立路径。

项目题的五层追问



同一个项目至少准备三层深度：90 秒概览、3 分钟机制解释、10 分钟证据链。若缺少真实数据，宁可说明观测方法和待验证假设，也不要编造结论。

03 面试前 30 分钟清单

- | | |
|----|---|
| 01 | 熟练讲解 capture / instantiate / replay 的生命周期。 |
| 02 | 准备一个算子的线程映射图与数据访问图。 |
| 03 | 复习 warp、block、shared memory、coalescing、occupancy。 |
| 04 | 写出正确性/性能基准的最小测试矩阵。 |

模拟练习规则

随机抽取一题，先用 20 秒复述问题与边界；再用 90 秒给出结构化答案；最后接受三轮“为什么 / 如何验证 / 何时失效”的追问。录音回听时，只检查逻辑、术语准确性和证据完整性，不保留个人信息。

本讲义只输出可复用的岗位知识与面试方法，不含个人隐私、逐字转写或反问内容。

小红书小冯别放弃